SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I **M.Sivam Kumar** **(192521017)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

M.Sivam kumar
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Q1. Online Banking Fund Transfer Testing

The repeated failures in the online banking system indicate that the existing testing strategy does not adequately focus on transaction boundaries and invalid inputs. In banking applications, even a small calculation or validation error can result in financial loss. Therefore, testing should be systematic, risk-based, and repeatable.

1. Equivalence Class Partitioning (ECP): The transfer amount can be divided into three classes: below the minimum limit, within the permitted range, and above the maximum limit. Assuming the allowed range is ₹100 to ₹50,000, representative values are ₹50, ₹500, and ₹60,000.

2. Boundary Value Analysis (BVA): Since defects frequently occur at comparison boundaries, test ₹99, ₹100, ₹101, ₹49,999, ₹50,000, and ₹50,001. These values help identify incorrect operators, such as using a strict greater-than condition instead of greater-than-or-equal-to.

3. Evaluation of Random Testing: Random testing may reveal unexpected input combinations, but it cannot guarantee that critical boundary values will be selected. It should therefore supplement, rather than replace, structured testing.

4. Optimized Testing Strategy: Combine ECP and BVA with tests for insufficient balance, invalid beneficiary details, transaction timeouts, and duplicate transfer requests. Verify that failed transactions do not deduct money and that successful transfers update both accounts consistently.

Conclusion: A risk-based test suite combining boundary-focused cases, equivalence classes, and selected random inputs offers better defect detection than random testing alone. It also makes test results repeatable and easier to audit.

Yahi style best rahega: scenario analysis + technique + specific test examples + justification + conclusion, bina unnecessary theory bharne ke.

Main baaki 14 answers bhi isi tarah thode detailed aur scenario-specific rakhunga. Marks ke liye content strong hoga, lekin 100 marks ki guarantee koi nahi de sakta.

Q2. Healthcare Application – Insufficient Test Coverage

The discovery of five critical defects after deployment indicates that the healthcare application was tested without adequately examining all important execution paths. In healthcare software, an incorrect result or an unhandled exception may affect clinical decisions, so testing must focus on both code coverage and risk.

1. Control Flow Graph (CFG): Consider a module that validates patient details, checks medical eligibility, processes a request, and displays the result. Its main flow is Start → Validate Details → Check Eligibility → Process Request → Display Result → End. Alternative paths must cover invalid patient details, failed eligibility, missing records, and processing errors.

2. Coverage Analysis: Statement coverage confirms that statements execute, but it may miss alternative decision outcomes. Branch coverage checks both true and false outcomes of each decision. Path coverage examines sequences of decisions, although complete path coverage may

be impractical when loops create many possible paths.

3. Improved Testing Strategy: Test valid and invalid patient records, eligible and ineligible patients, missing data, boundary values, and exception-handling paths. Measure statement and branch coverage and prioritize clinically critical paths.

Conclusion: Combining CFG-based testing with branch coverage and risk-based path selection improves reliability. It helps detect hidden logic errors before deployment and reduces the possibility of incorrect healthcare results.

Q3. Retail E-commerce – Input Combination Testing

The failure to detect errors involving price, quantity, and discount suggests that the existing testing strategy considered inputs individually rather than examining their combined effect. For example, a discount calculation may work correctly for a normal quantity but fail when the quantity is zero or the discount exceeds the permitted limit.

1. Equivalence Class Partitioning (ECP): Divide price into valid positive and invalid values, quantity into zero, positive, and negative values, and discount into permitted and excessive values. Select representative values from each class.

2. Boundary Value Analysis (BVA): Test the minimum permitted price, the smallest valid quantity, and discount values immediately below, at, and above the maximum allowed discount. These tests help identify rounding and comparison errors.

3. Decision Table Testing: Construct rules for combinations such as valid price with valid discount, valid price with excessive discount, and invalid quantity with a valid discount. Verify that each combination produces the expected total or an appropriate validation message.

4. Optimized Test Design: Use decision tables to cover important business rules and pairwise testing to reduce redundant combinations. Include checks for negative totals, incorrect rounding, and unauthorized discounts.

Conclusion: ECP, BVA, and decision table testing complement one another by covering input ranges, boundary defects, and interactions. This produces a more complete and efficient test suite than testing each field independently.

Q4. Software Module with Nested Conditions and Loops

Partial path coverage indicates that some execution routes through the nested conditions and loops have not been tested. If the control flow graph is incomplete, important branches, loop exits, and exception-handling routes may remain unnoticed.

1. Control Flow Graph Analysis: Represent each statement block or decision as a node and each possible transfer of control as an edge. Check that all true and false decision outcomes, loop-back edges, and loop-exit edges are included.

2. Independent Path Identification: Calculate cyclomatic complexity using:

$$V(G) = E - N + 2$$

Here, E represents the number of edges and N represents the number of nodes in a connected graph. The result estimates the number of linearly independent paths that should be considered

for basis path testing.

3. Test Design: Create test cases that execute independent paths and test loops with zero, one, and multiple iterations wherever applicable. Include nested-condition combinations and verify that every loop has a valid termination condition.

4. Expected Improvement: These tests can reveal incorrect loop conditions, unreachable branches, and errors that occur only when multiple conditions interact.

Conclusion: Basis path testing supported by a correctly constructed CFG improves structural coverage. Since loops may produce infinitely many paths, the practical objective is to test independent paths and all critical loop boundaries rather than claim exhaustive path coverage.

Q5. Financial Application – Hybrid Testing Strategy

The continued appearance of production defects suggests that the financial application's testing methods are not sufficiently integrated or that the selected coverage criteria do not reflect critical business risks. High code coverage alone cannot prove that transactions are financially correct.

1. Black-box Testing: Apply ECP and BVA to transaction amounts, account balances, interest rates, and withdrawal limits. For example, test a withdrawal amount immediately below, exactly equal to, and immediately above the available balance.

2. White-box Testing: Construct a CFG for authentication, balance validation, transaction calculation, account updating, and error handling. Apply basis path testing to examine independent execution routes and branch coverage to verify both outcomes of each decision.

3. Hybrid Test Design: Combine input-based cases with internal execution paths. Test valid transactions, insufficient funds, invalid account details, interrupted transactions, and repeated requests. Verify that a failed transaction does not create a debit and that retries do not cause duplicate payments.

4. Test Adequacy: Track statement and branch coverage alongside requirement coverage and transaction-level assertions. Prioritize tests according to financial impact rather than relying only on a percentage of executed code.

Conclusion: A hybrid strategy connects externally visible financial requirements with internal program logic. It improves defect detection, protects transaction consistency, and provides stronger evidence that the application is ready for production.

Q6. HR Payroll – Incorrect Bonuses at Service Milestones

Incorrect bonus payments at exactly 5, 10, and 15 years suggest a possible boundary-condition defect in the service-duration calculation. The program may incorrectly classify employees because of comparison operators, date calculations, or overlapping service categories.

1. Equivalence Class Partitioning: Assuming the policy assigns new bonus categories at these milestones, divide employees into service groups below 5 years, 5 to below 10 years, 10 to below 15 years, and 15 years or more. Test each category with representative employees.

2. Boundary Value Analysis: Test service durations immediately before, exactly at, and immediately after each milestone. For example, test 4 years 11 months, exactly 5 years, and 5 years 1 month. Repeat the process for 10- and 15-year thresholds.

3. Performance Rating Combinations: Since the bonus also depends on performance, test low, medium, and high ratings within each service category. Verify the expected payout using the approved payroll formula.

4. Why Structured Testing Is Better: Exploratory testing may discover unexpected problems but can overlook exact milestone values. ECP and BVA provide repeatable tests for every important category and boundary.

Conclusion: A milestone-focused test suite can expose incorrect comparisons and bonus calculations before payroll is processed. It also provides clear evidence that the implementation follows the intended compensation policy.

Q7. Laboratory Temperature Control – Alarm Failure

The failure to activate an alarm during a sensor malfunction suggests that the system's fault-handling path was either missing or insufficiently tested. Because this is a safety-critical controller, the testing strategy must examine individual Boolean conditions and the overall decisions they influence.

1. Conceptual CFG: Start → Read Sensor → Check Sensor Validity. If the sensor is invalid, activate the alarm and enter the safe state. Otherwise, check the temperature: activate cooling if it exceeds the upper limit, activate heating if it falls below the lower limit, and maintain the current state when the temperature is within range.

2. Branch Coverage: Test both valid and invalid sensor readings, high and low temperatures, and normal operating conditions. This confirms that each major decision outcome is executed.

3. Modified Condition/Decision Coverage (MC/DC): For each compound Boolean decision, demonstrate that every individual condition can independently change the decision outcome while other conditions are controlled. For example, if an alarm depends on sensor failure, excessive temperature, and a safety override, design cases that isolate each condition's influence.

4. Safety Validation: Test disconnected sensors, out-of-range readings, delayed sensor updates, and alarm activation. Hardware-in-the-loop testing can verify that the software actually triggers the required physical response.

Conclusion: MC/DC provides stronger evidence than statement coverage for complex safety logic. Combined with fault injection and hardware integration tests, it reduces the risk of an alarm failure during hazardous conditions.

Q8. Travel Booking Portal – Negative Total Amount

The negative total amount indicates that the booking portal may be combining promotional discounts without correctly checking their compatibility or enforcing a minimum payable amount. Testing each promotional code separately cannot reveal all defects caused by interactions.

1. Decision Table Testing: Identify conditions such as valid base price, promotion eligibility, customer tier, and whether multiple codes can be combined. Define the expected action for each meaningful combination.

2. Representative Rules:

- A valid price with one eligible code should receive the permitted discount.

- A premium user should receive a premium promotion only when its eligibility conditions are satisfied.
- Two compatible codes should be combined according to the promotion policy.
- Two incompatible codes should be rejected or handled using the defined priority rule.
- A discount greater than the base price must not produce a negative payable amount.

3. Boundary Testing: Test prices immediately below, at, and above relevant minimum values. Also test combinations that produce a zero balance or a discount close to the base price.

4. Optimization: Use a decision table to cover each business rule and pairwise testing for additional input interactions. This reduces redundant cases without ignoring important combinations.

Conclusion: Decision table testing reveals interaction defects that isolated tests miss. Explicit discount validation and total-amount assertions help prevent invalid charges and inconsistent booking records.

Q9. Legacy Database Migration – Infinite Loop Crashes

High statement coverage does not guarantee that a database migration tool terminates correctly. Every statement may execute at least once while a loop repeatedly processes the same record or fails to update its stopping condition.

1. Control Flow Analysis: Construct a CFG that identifies loop entry, record validation, record advancement, error handling, and loop exit. Examine whether every possible execution route can reach a valid termination point.

2. Cyclomatic Complexity: Calculate the number of independent paths using $V(G) = E - N + 2$. Use this value to guide basis path testing and identify important decision combinations.

3. Loop Boundary Tests: Test an empty database, one record, multiple valid records, invalid records, and the maximum supported dataset. Check zero iterations, one iteration, normal completion, and loop-exit conditions.

4. Progress and Termination Checks: Verify that the record index or iterator advances correctly after every iteration. Add appropriate timeouts and diagnostic logging to identify loops that exceed the expected processing duration.

Conclusion: Statement coverage measures execution, not termination. CFG analysis, independent path testing, and loop-boundary tests provide stronger evidence that the migration process will finish reliably without repeatedly processing records.

Q10. Food Delivery App – Real-time Tracker Stuck During Cancellation

The tracker becoming stuck when a driver cancels mid-delivery suggests that the application does not handle every event from every relevant state. State Transition Testing is suitable because the correctness of the application depends on both its current state and the event it receives.

1. State Model: Define the normal states as Ordered, Preparing, Out for Delivery, and Delivered. Introduce a Cancelled state and define cancellation rules for the stages where cancellation is permitted.

2. Transition Tests: Verify the normal sequence from Ordered to Delivered. Test cancellation during preparation and during delivery, according to the business rules. Each cancellation should lead to the correct status, customer notification, and refund workflow where applicable.

3. Invalid and Repeated Events: Attempt an invalid transition, such as moving directly from Ordered to Delivered. Also test repeated cancellation requests, delayed network responses, and duplicate status updates.

4. Expected Improvement: Record the expected state and output for each event. Verify that the tracker never remains indefinitely in an undefined state and that customer, driver, and order records remain consistent.

Conclusion: ECP and BVA primarily address input values, whereas State Transition Testing examines event sequences and state-dependent behavior. It is therefore more effective for finding workflow defects in real-time delivery systems.

Q11. Cybersecurity Authentication – Brute-force Vulnerability

The discovery of brute-force vulnerabilities indicates that authentication tests did not adequately examine repeated failures and interactions between password validation, OTP timing, and IP-based restrictions.

1. Black-box Testing: Test incorrect passwords, expired OTPs, repeated OTP submissions, and login attempts from unfamiliar IP addresses. Use error guessing to investigate likely weaknesses such as missing rate limits, OTP reuse, and ineffective account lockout.

2. White-box Path Analysis: Construct a CFG covering password verification, OTP validation, IP assessment, failed-attempt counting, and authorization. Test independent paths to confirm that access is never granted when a mandatory security condition fails.

3. Hybrid Test Strategy: Combine externally observable attack scenarios with internal path analysis. Verify rate limiting, temporary lockout, OTP expiration, secure error messages, and logging of suspicious activity.

4. Security Assertions: Confirm that repeated failed attempts cannot bypass controls and that successful authentication requires all mandatory checks. Also test whether retrying an interrupted request changes the authentication state incorrectly.

Conclusion: Error guessing targets realistic security weaknesses, while path analysis examines the internal logic that controls access. Their combination improves the detection of authentication defects and strengthens the module's security posture.

Q12. E-learning Platform – Subscription Upgrade on Expiration Date

The failure during an upgrade on the expiration date suggests that the system may use incorrect date comparisons or unclear rules for transferring access between subscription tiers.

1. Equivalence Class Partitioning: Divide subscription timing into three classes: before expiration, exactly on expiration, and after expiration. Divide subscription types into Free, Basic, and Premium, and identify the permitted upgrade combinations.

2. Date-Based Test Cases: Test an upgrade one day before expiration, on the expiration date, and

one day after expiration. Check whether access, payment, and course permissions follow the intended subscription policy.

3. Additional Boundary Conditions: Test month-end dates, leap days, timezone differences, and repeated upgrade requests. Verify that the subscription does not expire prematurely or grant unauthorized access.

4. Test Adequacy: Establish the exact rule for when an expiry date becomes effective. Then check each date class against the expected access level and transaction outcome.

Conclusion: ECP provides representative tests for subscription categories, while date-boundary testing targets errors around expiration. Together, they improve the reliability of tier changes and reduce interruptions in course access.

Q13. Smart-home Lighting – Branch and Condition Coverage

Relying only on statement coverage is risky because executing every statement does not prove that every switch-case outcome or Boolean condition has been evaluated. A lighting controller may work under normal conditions but fail when sensor readings and manual controls interact.

1. Branch Coverage: Identify all switch cases, default cases, and decision outcomes. Test low ambient light, normal light, bright ambient light, and invalid sensor readings, according to the algorithm's defined behavior.

2. Condition Coverage: For compound Boolean expressions, ensure each individual condition takes both true and false values where feasible. If the controller considers ambient light, occupancy, and manual override, test combinations that isolate each condition's effect.

3. Integration Testing: Check sensor disconnection, delayed readings, unexpected values, and the actual dimming command sent to the hardware. Confirm that invalid sensor data does not trigger unsafe or unpredictable output.

4. Improved Strategy: Combine branch and condition coverage with boundary-value tests for sensor thresholds. Use controlled hardware tests to compare the expected brightness level with the actual response.

Conclusion: Branch and condition coverage reveal logical gaps that statement coverage can miss. Adding sensor-fault and hardware-integration tests improves the reliability of the lighting controller in real-world conditions.

Q14. Logistics Management – Zero and Negative Shipping Weights

The failure to detect zero and negative weights indicates that input validation did not adequately test invalid boundaries. Incorrect weight acceptance can produce inaccurate shipping costs and inconsistent logistics records.

1. Robust Boundary Value Analysis: Assume the minimum accepted shipment weight is 0.1 kg. Test −0.1 kg, 0 kg, 0.09 kg, 0.1 kg, and 0.11 kg. These values distinguish invalid weights from the smallest permitted weight.

2. Additional Boundaries: Test values around the maximum supported weight and verify that rounding does not convert an invalid weight into a valid one. Repeat important checks across different destination zones and delivery speeds.

3. Expected Results: Negative and zero weights should be rejected with a clear validation message. Valid weights should produce the correct shipping charge, while excessive weights should follow the defined rejection or special-handling rule.

4. Importance of Negative Testing: Testing invalid inputs checks whether the application fails safely instead of continuing with incorrect data. It also helps prevent incorrect billing and downstream processing errors.

Conclusion: Robust BVA examines values on both sides of valid boundaries. This structured approach detects input-validation defects more reliably than testing only normal shipment weights.

Q15. Medical Imaging – Inconsistent Results for Prime-sized Datasets

The occurrence of inconsistent results only for prime-sized datasets suggests a defect in the algorithm's indexing, data partitioning, or parallel workload distribution. Random testing alone may not repeatedly select the dataset sizes needed to expose this specialized problem.

1. Basis Path Testing: Construct a CFG for the processing algorithm and calculate cyclomatic complexity using $V(G) = E - N + 2$. Identify independent execution paths involving loop initialization, data partitioning, parallel execution, and result merging.

2. Targeted Dataset Selection: Test empty and single-element datasets where supported, small prime sizes, larger prime sizes, and composite sizes. For example, compare inputs containing 7, 8, 11, and 12 elements to investigate whether the failure is linked to partition boundaries.

3. Accuracy Verification: Compare parallel-processing results with a trusted sequential implementation. Check indexing, missing elements, duplicated elements, and numerical differences within the defined tolerance.

4. Stability Testing: Repeat the same dataset multiple times and vary the distribution of work among processing threads. This helps reveal race conditions or errors that occur only under particular execution schedules.

Conclusion: Basis path testing provides a structured examination of independent logic routes, while prime-focused datasets directly target the reported failure. Together with reference-result comparison and repeated parallel execution, this strategy improves mathematical accuracy and system stability.